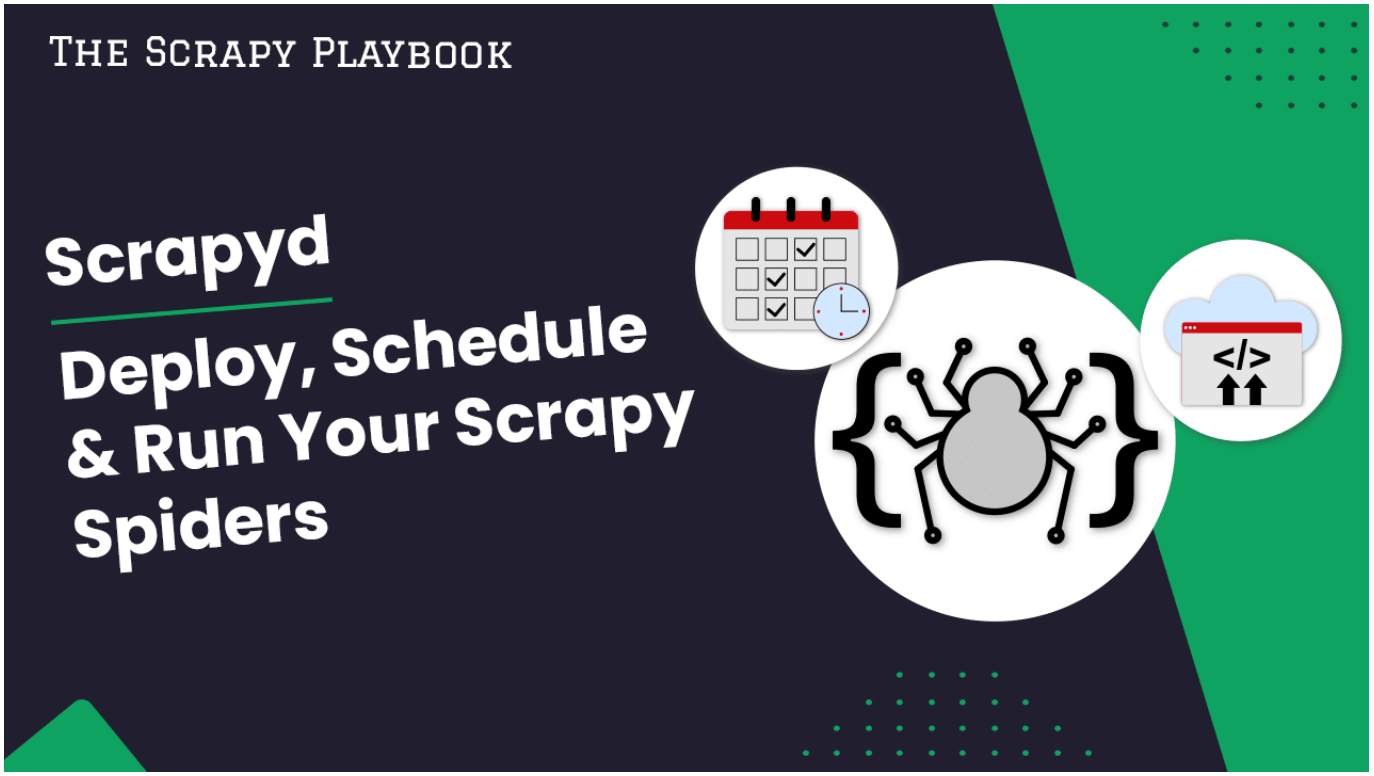


The Complete Guide To Scrapyd: Deploy, Schedule & Run Your Scrapy Spiders

You've built your scraper, tested that it works and now want to schedule it to run every hour, day, etc. and scrape the data you need. But what is the best way to do that?

Scrapyd is one of the most popular options. Created by the same developers that developed Scrapy itself, Scrapyd is a tool for running Scrapy spiders in production on remote servers so you don't need to run them on a local machine.

In this guide, we're going to run through:

- What Is Scrapyd?
- How To Setup Scrapyd?
- Deploying Spiders To Scrapyd
- Controlling Spiders With Scrapyd
- Integrating Scrapyd with ScrapeOps

There are many different Scrapyd dashboard and admin tools available, from ScrapeOps (Live Demo) to ScrapydWeb, SpiderKeeper, and more.

So if you'd like to choose the best one for your requirements then be sure to check out our Guide to the Best Scrapyd Dashboards, so you can see the pros and cons of each before you decide on which option to go with.

Need help scraping the web?

Then check out **ScrapeOps**, the complete toolkit for web scraping.

Proxy Manager **Scraper Monitoring** **Job Scheduling**

What Is Scrapyd?

Scrapyd is application that allows us to deploy Scrapy spiders on a server and run them remotely using a JSON API. Scrapyd allows you to:

- Run Scrapy jobs.
- Pause & Cancel Scrapy jobs.
- Manage Scrapy project/spider versions.
- Access Scrapy logs remotely.

Scrapyd is a great option for developers who want an easy way to manage production Scrapy spiders that run on a remote server.

With Scrapyd you can manage multiple servers from one central point by using a ready-made Scrapyd management tool like ScrapeOps, an open source alternative or by building your own.

Here you can check out the full Scrapyd docs and Github repo.

If you prefer video tutorials, then check out our tutorial on how to setup Scrapyd and integrate it with ScrapeOps & Digital Ocean.

How to Setup Scrapyd

Getting Scrapyd setup is quick and simple. You can run it locally or on a server.

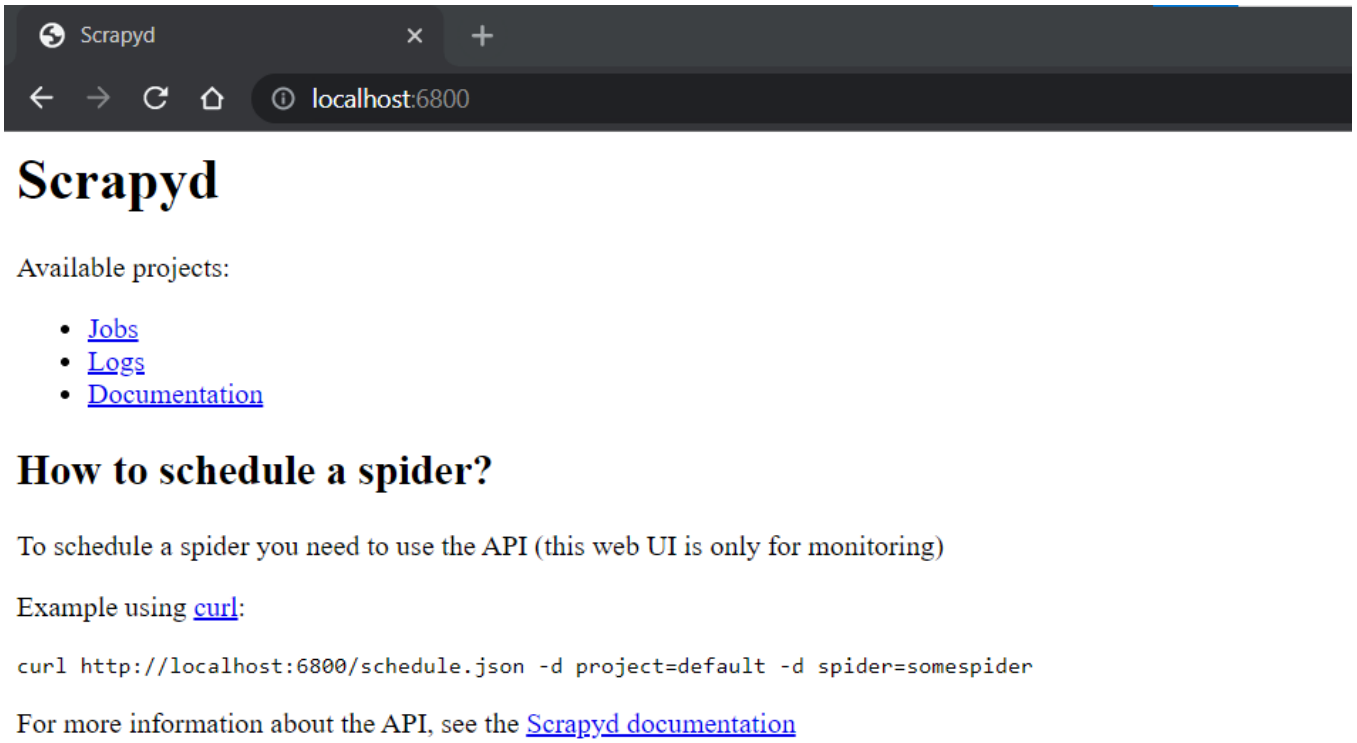
First step is to install Scrapyd:

pip install scrapyd

And then start the server by using the command:

scrapyd

This will start Scrapyd running on `http://localhost:6800/`. You can open this url in your browser and you should see the following screen:



Deploying Spiders To Scrapyd

To run jobs using Scrapyd, we first need to eggify and deploy our Scrapy project to the Scrapyd server. To do this, there is a easy to use library called scrapyd-client that makes this process very simple.

First let's install scrapyd-client

pip install git+https://github.com/scrapy/scrapyd-client.git

Once installed, navigate to your Scrapy project you want to deploy and open your `scrapyd.cfg` file, which should be located in your projects root directory. You should see something like this, with the **"demo"** text being replaced by your Scrapy projects name:

scrapy.cfg

[settings] default = demo.settings

[deploy] #url = http://localhost:6800/ project = demo

Here the `scrapyd.cfg` configuration file defines the endpoint your Scrapy project should be be deployed to. To enable us to deploy our project to Scrapyd, we just need to uncomment the `url` value if we want to deploy it to a locally running Scrapyd server.

scrapy.cfg

[settings] default = demo.settings

[deploy] url = http://localhost:6800/ project = demo

Then run the following command in your Scrapy projects root directory:

scrapyd-deploy default

This will then eggify your Scrapy project and deploy it to your locally running Scrapyd server. You should get a result like this in your terminal if it was successful:

```
    $ scrapyd-deploy default
    Packing version 1640086638
    Deploying to project "demo" in http://localhost:6800/addversion.json
    Server response (200):
    {"node_name": "DESKTOP-67BR2", "status": "ok", "project": "demo", "version":
"1640086638", "spiders": 1}
```

Now your Scrapy project has been deployed to your Scrapyd and is ready to be run.

Aside: Custom Deployment Endpoints

The above example was the simplest implementation and assumed you were just deploying your Scrapy project to a local Scrapyd server. However, you can customise or add multiple deployment endpoints to `scrapyd.cfg` file if you would like.

For example you can define local and production endpoints:

```
    ## scrapy.cfg

    [settings]
    default = demo.settings

    [deploy:local]
    url = http://localhost:6800/
    project = demo

    [deploy:production]
    url = <MY_IP_ADDRESS>
    project = demo
```

And deploy your Scrapy project locally or to production using this command:

```
## Deploy locally
scrapyd-deploy local

## Deploy to production
scrapyd-deploy production
```

Or deploy a specific project using by specifying the project name:

```
scrapyd-deploy <target> -p <project>
```

For more information about this, check out the scrapyd-client docs here.

Controlling Spiders With Scrapyd

Scrapyd comes with a minimal web interface which can be accessed at http://localhost:6800/, however, this interface is just a rudimentary overview of what is running on a Scrapyd server and doesn't allow you to control the spiders deployed to the Scrapyd server.

To control your spiders with Scrapyd you have 3 options:

- Scrapyd JSON API
- Python-Scrapyd-API Library
- Scrapyd Dashboard

Scrapyd JSON API

To schedule, run, cancel jobs on your Scrapyd server we need to use the JSON API it provides. Depending on the endpoint, the API supports `GET` or `POST` HTTP requests. For example:

```
$ curl http://localhost:6800/daemonstatus.json
{ "status": "ok", "running": "0", "pending": "0", "finished": "0", "node_name": "DESKTOP-67BR2" }
```

The API has the following endpoints:

Endpoint	Description
daemonstatus.json	Checks the status of the Scrapyd server.
addversion.json	Add a version to a project, creating the project if it doesn't exist.
schedule.json	Schedule a job to run.

Endpoint	Description
cancel.json	Cancel a job. If the job is pending, it will be removed. If the job is running, the job will be shutdown.
listprojects.json	Returns a list of the projects uploaded to the Scrapyd server.
listversions.json	Returns a list of versions available for the requested project.
listspiders.json	Returns a list of the spiders available for the requested project.
listjobs.json	Returns a list of pending, running and finished jobs for the requested project.
delversion.json	Deletes a project version. If project only has one version, deletes the project too.
delproject.json	Deletes the project, and all associated versions.

Full API specifications can be found here.

We can interact with these endpoints using **Python Requests** or any other HTTP request library, or we can use python-scrapyd-api a Python wrapper for the Scrapyd API.

Python-Scrapyd-API Library

The python-scrapyd-api provides a clean and easy to use Python wrapper around the Scrapyd JSON API, which can simplify your code.

First, we need to install it:

pip install python-scrapyd-api

Then in our code we need to import the library and configure it to interact with our Scrapyd server by passing it the Scrapyd IP address.

> from scrapyd_api import ScrapydAPI scrapyd = ScrapydAPI('http://localhost:6800')

From here, we can use the built in methods to interact with the Scrapyd server.

Check Daemon Status

Checks the status of the Scrapyd server.

daemon_status may be deprecated, attribute is no longer available

```
>>> scrapyd.daemon_status()
{u'finished': 0, u'running': 0, u'pending': 0, u'node_name': u'DESKTOP-67BR2'}
```

List All Projects

Returns a list of the projects uploaded to the Scrapyd server.

```
>>> scrapyd.list_projects()
[u'demo', u'quotes_project']
```

List All Spiders

Enter the project name, and it will return a list of the spiders available for the requested project.

```
>>> scrapyd.list_spiders('project_name')
[u'raw_spider', u'js_enhanced_spider', u'selenium_spider']
```

Run a Job

Run a Scrapy spider by specifying the project and spider name.

```
>>> scrapyd.schedule('project_name', 'spider_name')
# Returns the Scrapyd job id.
u'14a6599ef67111e38a0e080027880ca6'
```

Pass custom settings using the `settings` arguement.

```
>>> settings = {'DOWNLOAD_DELAY': 2}
>>> scrapyd.schedule('project_name', 'spider_name', settings=settings)
u'25b6588ef67333e38a0e080027880de7'
```

One important thing to note about the schedule.json API endpoint. Even though the endpoint is called schedule.json, using it only adds a job to the internal Scrapy scheduler queue, which will be run when a slot is free.

This endpoint doesn't have the functionality to schedule a job in the future so it runs at specific time, Scrapyd will add the job to a queue and run it once a Scrapy slot becomes available.

To actually schedule a job to run in the future at a specific date/time or periodicially at a specific time then you will need to control this scheduling on your end. Tools like ScrapeOps will do this for you.

Cancel a Running Job

Cancel a running job by sending the project name and the job_id.

```
>>> scrapyd.cancel('project_name', '14a6599ef67111e38a0e080027880ca6')
# Returns the "previous state" of the job before it was cancelled: 'running' or 'pending'.
'running'
```

When sent it will return the "previous state" of the job before it was cancelled. You can verify that the job was actually cancelled by checking the jobs status.

```
>>> scrapyd.job_status('project_name', '14a6599ef67111e38a0e080027880ca6')
# Returns 'running', 'pending', 'finished' or '' for unknown state.
'finished'
```

For more functionality then check out the **python-scrapyd-api** documentation here.

Scrapyd Dashboard

Using Scrapyd's JSON API to control your spiders is possible, however, it isn't ideal as you will need to create custom workflows on your end to monitor, manage and run your spiders. Which can become a major project in itself if you need to manage spiders spread across multiple servers.

Other developers ran into this problem so luckily for us, they decided to create free and open-source Scrapyd dashboards that can connect to your Scrapyd servers so you can manage everything from a single dashboard.

There are many different Scrapyd dashboard and admin tools available:

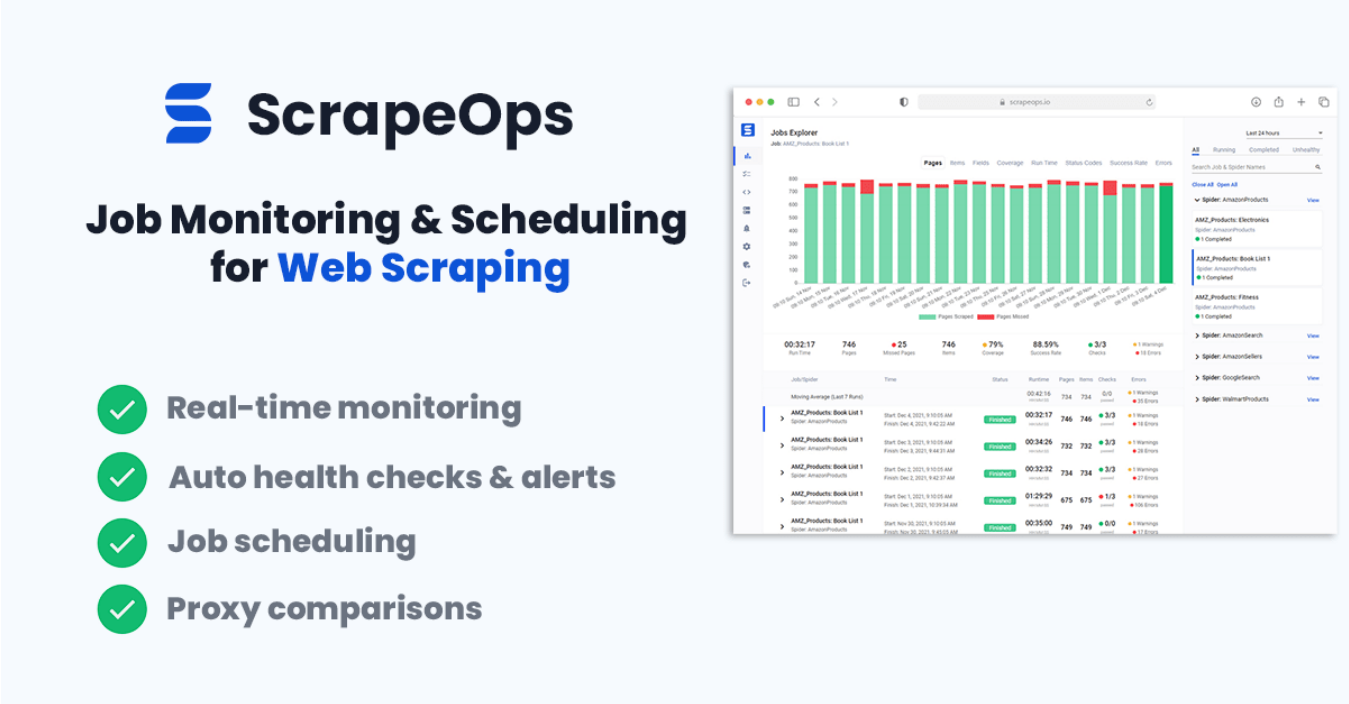
- ScrapeOps (Live Demo)
- ScrapydWeb
- SpiderKeeper

If you'd like to choose the best one for your requirements then be sure to check out our Guide to the Best Scrapyd Dashboards here.

Integrating Scrapyd with ScrapeOps

ScrapeOps is a free monitoring tool for web scraping that also has a Scrapyd dashboard that allows you to schedule, run and manage all your scrapers from a single dashboard.

Live demo here: ScrapeOps Demo



With a simple 30 second install ScrapeOps gives you all the monitoring, alerting, scheduling and data validation functionality you need for web scraping straight out of the box.

Unlike the other Scrapyd dashboard, ScrapeOps is a full end-to-end web scraping monitoring and management tool dedicated to web scraping that automatically sets up all the monitors, health checks and alerts for you.

Features

Once setup, ScrapeOps will:

- 🕵️ **Monitor -** Automatically monitor all your scrapers.
- 📈 **Dashboards -** Visualise your job data in dashboards, so you see real-time & historical stats.
- 💯 **Data Quality -** Validate the field coverage in each of your jobs, so broken parsers can be detected straight away.
- 📉 **Auto Health Checks -** Automatically check every jobs performance data versus its 7 day moving average to see if its healthy or not.
- ✔️ **Custom Health Checks -** Check each job with any custom health checks you have enabled for it.
- ⏰ **Alerts -** Alert you via email, Slack, etc. if any of your jobs are unhealthy.
- 📑 **Reports -** Generate daily (periodic) reports, that check all jobs versus your criteria and let you know if everything is healthy or not.

Integration

There are two steps to integrate ScrapeOps with your Scrapyd servers:

- Install ScrapeOps Logger Extension

- Connect ScrapeOps to Your Scrapyd Servers

Note: You can't connect ScrapeOps to a Scrapyd server that is running locally, and isn't offering a public IP address available to connect to.

Once setup you will be able to schedule, run and manage all your Scrapyd servers from one dashboard.

![ScrapeOps Dashboard Demo]((https://assets-scrapeops.nyc3.digitaloceanspaces.com/Images%2FScrapeOps_Assets%2Fscrapeops-scheduler-holder.png)

Step 1: Install Scrapy Logger Extension

For ScrapeOps to monitor your scrapers, create dashboards and trigger alerts you need to install the ScrapeOps logger extension in each of your Scrapy projects.

Simply install the Python package:

pip install scrapeops-scrapy

And add 3 lines to your `settings.py` file:

```
## settings.py

## Add Your ScrapeOps API key
SCRAPEOPS_API_KEY = 'YOUR_API_KEY'

## Add In The ScrapeOps Extension
EXTENSIONS = {
  'scrapeops_scrapy.extension.ScrapeOpsMonitor': 500,
}

## Update The Download Middlewares
DOWNLOADER_MIDDLEWARES = {
    'scrapeops_scrapy.middleware.retry.RetryMiddleware': 550,
    'scrapy.downloadermiddlewares.retry.RetryMiddleware': None,
}
```

From there, your scraping stats will be automatically logged and automatically shipped to your dashboard.

Step 2: Connect ScrapeOps to Your Scrapyd Servers

The next step is giving ScrapeOps the connection details of your Scrapyd servers so that you can manage them from the dashboard.

Enter Scrapyd Server Details

Within your dashboard go to the Servers page and click on the **Add Scrapyd Server** at the top of the page.

In the dropdown section then enter your connection details:

- Server Name
- Server Domain Name (optional)
- Server IP Address

Whitelist Our Server (Optional)

Depending on how you are securing your Scrapyd server, you might need to whitelist our IP address so it can connect to your Scrapyd servers. There are two options to do this:

Option 1: Auto Install (Ubuntu)

SSH into your server as root and run the following command in your terminal.

```
wget -O scrapeops_setup.sh "https://assets-scrapeops.nyc3.digitaloceanspaces.com/
Bash_Scripts/scrapeops_setup.sh"; bash scrapeops_setup.sh
```

This command will begin the provisioning process for your server, and will configure the server so that Scrapyd can be managed by Scrapeops.

Option 2: Manual Install

This step is optional but needed if you want to run/stop/re-run/schedule any jobs using our site. If we cannot reach your server via port 80 or 443 the server will be listed as read only.

The following steps should work on Linux/Unix based servers that have UFW firewall installed.:

Step 1: Log into your server via SSH

Step 2: Enable SSH'ing so that you don't get blocked from your server

```
sudo ufw allow ssh
```

Step 3: Allow incoming connections from 46.101.44.87

```
sudo ufw allow from 46.101.44.87 to any port 443,80 proto tcp
```

Step 4: Enable ufw & check firewall rules are implemented

```
sudo ufw enable
sudo ufw status
```

Step 5: Install Nginx & setup a reverse proxy to let connection from scrapeops reach your scrapyd server.

```
sudo apt-get install nginx -y
```

Add the proxy_pass & proxy_set_header code below into the "location" block of your nginx default config file (default file usually found in /etc/nginx/sites-available)

```
proxy_pass http://localhost:6800/;
proxy_set_header X-Forwarded-Proto http;
```

Reload your nginx config

```
sudo systemctl reload nginx
```

Once this is done you should be able to run, re-run, stop, schedule jobs for this server from the ScrapeOps dashboard.

More Scrapy Tutorials

That's it for how to use Scrapyd to run your Scrapy spiders. If you would like to learn more about Scrapy, then be sure to check out The Scrapy Playbook.